

Guía práctica de comandos Docker

Parar e eliminar contenedores

```
# Parar todos os contedores actualmente en execución
docker stop $(docker ps -a -q)

# Eliminar todos os contedores (mesmo os parados)
docker rm -f $(docker ps -a -q)

# Eliminar todas as imaxes dispoñibles no sistema
docker rmi -f $(docker images -a -q)
```

Traballar con imaxes

Sitios útiles:

- [Docker Hub](#)
- [dive](#) (inspección de capas)

Buscar imaxes

```
docker images                # Lista de imaxes descargadas localmente
docker search nginx          # Busca imaxes públicas con ese nome
docker search --help         # Amosar axuda para o comando search
docker search -f is-official=true nginx # Busca só imaxes oficiais
docker search --filter=stars=20 nginx   # Imaxes con máis de 20 estrelas
docker search adbgonzalez          # Busca imaxes dun usuario concreto
```

Descargar imaxes

```
docker pull nginx            # Descarga a última versión de Nginx
docker pull httpd            # Descarga a imaxe de Apache HTTP Server
docker pull mysql:5.7        # Descarga versión específica de MySQL
docker pull mysql            # Descarga a última versión dispoñible
docker pull adbgonzalez/kafka:3.9 # Descarga imaxe personalizada dun usuario
```

Inspección de imaxes

```
docker history nginx        # Amosar as capas dunha imaxe
docker inspect nginx        # Información detallada (JSON)
docker rmi nginx            # Eliminar unha imaxe
```

```
docker rmi -f nginx          # Forzar a eliminación
docker rmi --help            # Axuda sobre o comando rmi
```

Ruta local de almacenamento de imaxes:

```
/var/lib/docker/image
```

Lanzar contedores (run/create)

```
docker run --help            # Axuda sobre o comando run
docker create --help         # Axuda sobre o comando create

docker run -dti httpd        # Crea e inicia un contedor con Apache (modo
interactivo + terminal)
docker run -dti --rm --name web1 httpd  # Contedor efémero con nome "web1"
docker run -dti --name web-2 httpd echo hola  # Executa un comando específico

docker create --rm --name web-create httpd  # Crea un contedor sen arrincar
docker start web-create                  # Inicia o contedor creado

docker container run -dti --rm --name web3 httpd  # Versión estendida do comando
run
docker run -dti --name web2 nginx                # Arrinca un servidor Nginx
docker run -dti nginx                            # Arrinca Nginx sen nome específico
docker run -dti --name bd2 -e MYSQL_ROOT_PASSWORD=000000 mysql  # Arrinca unha BD
MySQL con contrasinal definido

# Exemplo que fallará por non establecer contrasinal de root
docker run -dti --name bd3 mysql
```

Acceder ao contedor

```
docker exec -ti bd2 /bin/bash          # Entrar no contedor "bd2" cunha shell
# Unha vez dentro:
mysql -u root -p                      # Acceder á base de datos MySQL
```

```
docker exec -ti bd3 /bin/bash          # Intentar acceder a outro contedor
docker inspect web1                   # Ver detalles do contedor "web1"
```

Ver contedores

```
docker ps -a                # Amosar todos os contedores (activos e
parados)
docker ps -l                # Amosar o último contedor creado
docker container ls --help  # Axuda para o comando ls
docker container ls --all   # Igual que “docker ps -a”
```

Interacción e xestión de contedores

```
docker inspect web1        # Información detallada dun contedor
docker exec -ti web1 /bin/bash  # Acceder á shell dun contedor activo
docker stop nome-contenedor  # Parar un contedor
docker start nome-contenedor  # Arrincar un contedor parado
docker restart nome-contenedor # Reiniciar un contedor
docker pause nome-contenedor  # Pausar procesos dun contedor
docker unpause nome-contenedor # Retomar execución
docker rm nome-contenedor     # Eliminar un contedor
docker rm -f nome-contenedor  # Forzar eliminación
docker logs nome-contenedor   # Amosar logs do contedor
docker logs -f nome-contenedor # Seguir logs en tempo real
docker logs -t nome-contenedor # Amosar logs con marcas de tempo
docker top contenedor1       # Ver procesos activos dentro dun contedor
exit                          # Saír da shell dun contedor
```

Estatísticas e actualización de contedores

```
docker stats                # Amosar consumo de recursos dos contedores
docker update --help        # Axuda para cambiar recursos dun contedor
docker update -m 512m --memory-swap -1 web1 # Limita a memoria do contedor web1
a 512MB
```

Opcións de `docker update`

Usage: `docker update [OPTIONS] CONTAINER [CONTAINER...]`

Update configuration of one or more containers

Options:

<code>--blkio-weight uint16</code>	Block IO weight (10–1000)
<code>--cpu-period int</code>	CPU CFS period
<code>--cpu-quota int</code>	CPU CFS quota
<code>--cpu-rt-period int</code>	Real-time period
<code>--cpu-rt-runtime int</code>	Real-time runtime
<code>-c, --cpu-shares int</code>	CPU shares

--cpus decimal	Number of CPUs
--cpuset-cpus string	Allowed CPUs (e.g. 0-3)
--cpuset-mems string	Allowed memory nodes
--kernel-memory bytes	Kernel memory limit
-m, --memory bytes	Memory limit
--memory-reservation bytes	Soft memory limit
--memory-swap bytes	Swap limit ('-1' = unlimited)
--pids-limit int	PIDs limit (-1 = unlimited)
--restart string	Restart policy

Exemplo de uso:

```
docker run -dti --name web1 httpd      # Crea contedor
docker ps -a                          # Comproba que está en execución
docker update -m 256m --memory-swap -1 web1 # Limita a 256 MB de RAM
```

Políticas de reinicio

```
--restart:

off          → Non reiniciar
on-failure   → Reiniciar só en caso de erro
always       → Reiniciar sempre que se pare
unless-stopped → Reiniciar salvo que se detivese explicitamente
```

Exemplo:

```
docker run -dtiP --name web5 --restart always nginx
docker run -dti -p 8084:80 --name web1 --restart unless-stopped httpd
```

Outras opcións útiles

```
--rm      → Elimina o contedor ao deterse automaticamente
-w        → Define o directorio de traballo dentro do contedor
-u        → Define o usuario que executa os comandos no contedor
```

Publicar servizos e NAT de portos

```
docker run -dtiP --name web11 httpd # Publica automaticamente os portos
docker run -dti -p 8085:80 --name web2 nginx # Expón porto 8085 do host
docker run -dti -p 3307:3306 --name bd2 -e MYSQL_ROOT_PASSWORD=000000 mysql #
Exposición manual de porto MySQL

iptables -t nat -L DOCKER -v -n # Ver regras de NAT creadas por Docker
docker ps -a # Ver portos asignados
docker port web11 # Consultar portos expostos dun contedor
```

Accede desde o navegador:

```
http://192.168.33.10:PUERTO
```

```
-P → Publica todos os portos dispoñibles de forma automática
-p → Publicación manual dun porto (erro se xa está en uso)
```

Exposición de rangos de portos

```
EXPOSE 7000-8000 # Declaración no Dockerfile
```

```
docker run --expose=7000-8000 # Exposición sen mapeo externo
docker run -p 7000-8000:7000-8000 imaxe-docker # Mapeo directo host ↔ contedor
```

Lanzamento de contedores

Estes comandos permiten crear e executar contedores de forma rápida a partir de imaxes existentes.

```
# Crea e executa un contedor en segundo plano (modo detached)
# O parámetro -P publica automaticamente todos os portos expostos
docker run -dtiP --name web2-lab httpd

# Crea un contedor de Nginx, publicando manualmente o porto 81 do host ao 80 do
contedor
docker run -dti --name web3 -p 81:80 nginx
```

Ver contedores en execución

```
docker ps -a          # Amosar todos os contedores (activos e detidos)
iptables -t nat -L DOCKER -v -n # Ver as regras de NAT creadas por Docker
```

IP de exemplo do contedor:

```
192.168.33.10
```

Copias de seguridade de imaxes

As copias de seguridade permiten exportar e gardar imaxes localmente para reutilizalas ou transportalas.

```
docker images | grep httpd      # Comprobar se a imaxe httpd existe
docker save -o httpd.tar httpd  # Gardar a imaxe httpd nun ficheiro TAR
docker rmi httpd                # Eliminar a imaxe localmente
docker images                   # Verificar que xa non aparece na lista
```

Importar copia de seguridade

Para restaurar unha imaxe previamente gardada:

```
docker load -i httpd.tar      # Cargar imaxe desde o ficheiro TAR
docker images | grep httpd    # Confirmar que a imaxe foi importada
```

Converter un contedor nunha imaxe

Este proceso permite xerar unha nova imaxe a partir dun contedor xa configurado manualmente.

```
# Crear un contedor base CentOS con stack LAMP
docker run -dtip --name centos6 docker.io/nickistre/centos-lamp

# Acceder ao contedor para personalizalo (por exemplo instalar Joomla)
docker exec -ti centos6 /bin/bash

# Crear unha nova imaxe a partir do contedor modificado
docker commit -m "Intranet con Joomla" centos6 intranet-martes31

# Crear un novo contedor desde a imaxe resultante
docker run -ditP --name intranet1 intranet-martes31
```

Exemplo de saída do comando `docker ps -l`:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
f0bd287fae86	intranet	"supervisord -n"	7 seconds ago	Up 6 seconds	
0.0.0.0:32774->22/tcp, 0.0.0.0:32773->80/tcp					

Etiquetar e executar unha imaxe

As etiquetas (*tags*) permiten versionar e identificar mellor as imaxes.

```
docker tag intranet intranet-martes31:v1      # Engadir etiqueta v1 á imaxe
docker run -ditP --name intranet1 intranet-martes31:v1  # Executar a versión
etiquetada
```

Volumes montados

Os volumes permiten persistir datos (por exemplo, BDs e ficheiros web) fóra do contedor.

```
# Montar directorios locais como volumes
docker run -ditP --name intranet1 -v /bd2:/var/lib/mysql -v /web2:/var/www/html
intranet-martes31

# Alternativa: empregar volumes nomeados xestionados por Docker
docker run -ditP --name intranet1 -v bd2:/var/lib/mysql -v web2:/var/www/html
intranet-martes31
```

- `/bd2` ou `bd2`: volume para datos de base de datos MySQL.
- `/web2` ou `web2`: volume para ficheiros da aplicación web.

Publicar unha imaxe en Docker Hub

Para subir unha imaxe ao repositorio público de DockerHub é necesario ter unha conta e iniciar sesión.

```
# Engadir unha etiqueta co formato usuario/imaxe
docker tag intranet-lunes19:v1 usuario/intranet

# Iniciar sesión co usuario de DockerHub
docker login

# Subir a imaxe ao repositorio
docker push usuario/intranet

# Pechar sesión
docker logout
```

Outros comandos útiles

```
docker pull usuario/intranet           # Descargar unha imaxe publicada
docker tag intranet miusuariodockerhub/intranet # Renomear antes de subir
cat /root/.docker/config.json          # Comprobar credenciais gardadas
docker search usuario                  # Buscar imaxes dun usuario
docker search miusuariodockerhub       # Buscar imaxes doutro usuario
```

Publicar un Nginx personalizado

Neste exemplo publícase unha imaxe modificada de Nginx co nome do usuario.

```
docker tag nginx usuario/nginx-lab    # Etiquetar a imaxe co nome do usuario
docker login                          # Iniciar sesión
cat /root/.docker/config.json          # Verificar credenciais
docker push usuario/nginx-lab          # Subir a imaxe personalizada
docker logout                         # Pechar sesión
docker search usuario                  # Confirmar que aparece no Docker Hub
```

Traballo con Volumes en Docker

Referencia oficial: [Volumes en Docker](#)

Un volume en Docker é un **directorio especial persistente** asignado a un contedor.

Os volumes permiten **gardar datos de forma independente ao ciclo de vida dos contedores**.

Localízanse normalmente en:

```
/var/lib/docker/volumes
```

Comandos básicos de volume

Docker ofrece un subcomando específico para xestionar volumes: `docker volume`.

```
docker volume

Usage:  docker volume COMMAND

Manage volumes

Commands:
  create      Create a volume
  inspect     Display detailed information on one or more volumes
```


ls	List volumes
prune	Remove all unused local volumes
rm	Remove one or more volumes

Listar os volumes dispoñibles:

```
docker volume ls
```

Exemplo de saída:

DRIVER	VOLUME NAME
local	0bf977f8de99b90fa1f2d7250415700cad4a6aaeff80c1e0c7bbd934aecaac19

Uso de **-v** ou **--mount**

Ambas opcións permiten montar volumes, pero **--mount** é máis explícita e moderna.

Iniciar un contedor con volume

```
# Usando --mount
docker run -d --name devtest --mount source=myvol2,target=/app nginx:latest

# Usando -v (forma abreviada)
docker run -d --name devtest -v myvol2:/app nginx:latest
```

💡 Ambos crean o volume automaticamente se non existe.

Laboratorio: WordPress + MySQL con volume persistente

Neste exemplo, creamos un pequeno laboratorio con dous servizos: unha base de datos e WordPress.

```
# Parar e eliminar contedores existentes
docker stop $(docker ps -a -q)
docker rm -f $(docker ps -a -q)

# Crear directorio para persistencia de datos
mkdir /bd

# Lanzar contedor MySQL con volume persistente
docker run -dti --name servidor_mysql -e MYSQL_ROOT_PASSWORD=000000 -v
/bd:/var/lib/mysql mysql:5.7
```

```
# Lanzar WordPress conectado a MySQL
docker run -dti --name servidor_wp -p 8000:80 --link servidor_mysql:mysql
wordpress:5.6.2-php7.3
```

Acceso desde o navegador:

```
http://192.168.33.10:8000/
```

Primeira forma: Volumes automáticos

Cando non se especifica unha ruta local, Docker crea volumes automáticos.

```
docker run -dtiP --name centos6-lamp -v /var/www/html docker.io/nickistre/centos-
lamp
docker inspect centos6-lamp
docker volume inspect <nome_volume>
```

Ruta típica dun volume:

```
/var/lib/docker/volumes/<nome_volume>/_data
```

Segunda forma: Volume ligado a un directorio local

Esta opción é ideal para desenvolver, xa que permite editar ficheiros directamente no host.

```
mkdir /web
cd /web
echo "proba de volume" > index.html

# Montar directorio local con acceso total
docker run -dtiP --name centos6-prueba-web -v /web:/var/www/html
docker.io/nickistre/centos-lamp

# Montar o mesmo directorio en modo lectura
docker run -dtiP --name centos6-prueba-web2 -v /web:/var/www/html:ro
docker.io/nickistre/centos-lamp
```

- `/web` → Directorio do host con contido web.
- `/var/www/html` → Ruta onde o servidor web serve os ficheiros.

Terceira forma: Volume nomeado + volume ligado

Combina volumes persistentes creados por Docker con directorios do sistema anfitrión.

```
docker volume create --name webapp      # Crear volume nomeado
docker volume ls                        # Ver lista de volumes
docker volume inspect webapp            # Ver detalles
```

Engadir contido manualmente ao volume:

```
echo "proba de volume" > /var/lib/docker/volumes/webapp/_data/index.html
```

Lanzar contedor con ambos volumes:

```
mkdir /web
docker run -dtip --name centos6-pruebacreacion3 -v webapp:/var/www/html -v
/web:/datos docker.io/nickistre/centos-lamp
docker inspect centos6-pruebacreacion3 | grep -A 5 -i mounts
docker exec -ti centos6-pruebacreacion3 /bin/bash
ls /datos
```

Eliminar contedores detidos (en estado *Exited*):

```
docker rm $(docker ps -a -f status=exited -q)
```

Exemplo con `--mount`

```
# Crear contedor con volume mediante --mount
docker run -d --name devtest --mount source=myvol2,target=/app nginx:latest

# Ver volumes dispoñibles e inspeccionar
docker volume ls
docker volume inspect myvol2
```

Comprobar montaxe dentro do contedor:

```
docker exec -ti devtest bash
df -h    # Ver volumes montados
```

Volume en modo lectura

Permite montar un volume só para lectura, impedindo modificacións no contedor.

```
# Forma longa
docker run -d --name=nginxtest --mount source=nginx-
vol,destination=/usr/share/nginx/html,readonly nginx:latest

# Forma abreviada
docker run -d --name=nginxtest -v nginx-vol:/usr/share/nginx/html:ro nginx:latest
```

Explicación:

- **source** → Nome do volume.
- **destination** → Ruta dentro do contedor.
- **readonly** → Indica montaxe en modo só lectura.